

RTC tervezése

IT eszközök technológiája IMSC feladat

Terv: századmásodperc – óra számolása folytonosan, és mindig a szükséges megjelenítése.

A legkisebb számláló a századmásodperc. A rendszer órajelet ennek megfelelően 100 Hz-re tervezem, Ami állapotgép segítségével működik.

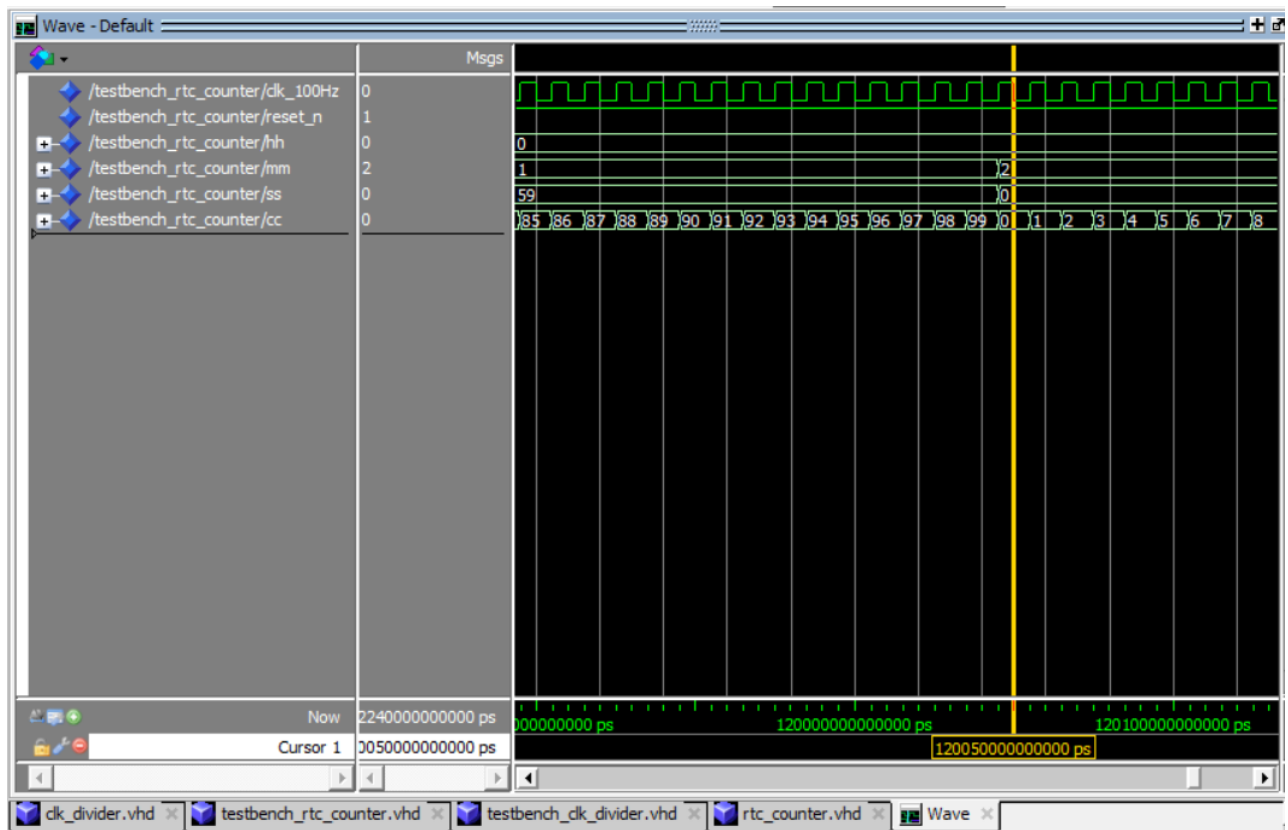
A kimeneten az idő hh:mm vagy ss:cc formátumban jelenik meg, ez nyomógombbal váltható.

HDL

Mivel a DE0 board 50 MHz-es órajelet állít elő, ezért készítettem egy órajelosztó modult `clk_divider` néven, ami 100 Hz-eset fog.

Az `rtc_counter` modulom tartja számon az időt. A számlálóim közül a századmásodpercnak 0-tól 99-ig, a másodperc és percnak 0-tól 59-ig, az órának 0-tól 23-ig kell számolnia és mindegyiknek akkor kell növekednie, ha az előző elérte a felső limitet.

Így egységesen tudom kezelni, hogy a többi modult csak akkor léptetem, ha az előző elérte a saját limitét.



testbench_rtc_counter eredménye

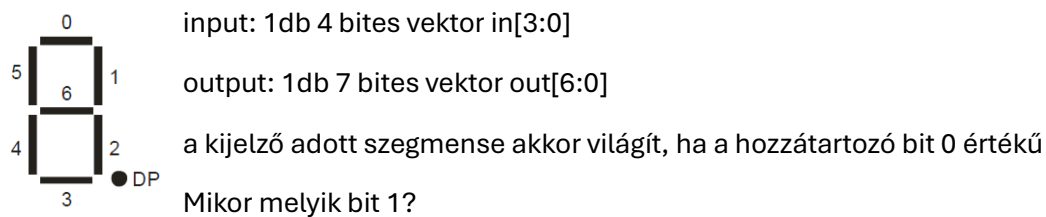
Az újrahasználatosság érdekében lehetne a laboron készített bcd számlálót módosítani, hogy megadott értékig számoljon el, majd ezeket a tc és en megfelelő összeköttetésével példányosítva megkaphatom mindegyik időegységem mindkét számjegyének számlálóját.

Példa: HH_H 2 <- HH_L 9 <- MM_H <- 5 <- MM_L 9 <- SS_H 5 <- SS_L 9 <- CC_H 9 <- CC_L 9

Megjegyzés: HH_L-t ki kell még egészíteni +1 bemenettel, amit HH_H-től kap, hogy ha 2 az értéke, akkor HH_L csak 3-ig számláljon.

Azért nem így valósítottam meg a feladatot, mert jelen esetben sokkal egyszerűbb egy modulban végezni minden időegység számolását, majd ezt top level szinten BCD számjegyekre bontani a hétszegmenses kijelzős megjelenítéshez.

A hétszegmenses kijelző implementálása a laborral megegyező hex.vhd:



0->6 1->6,5,4,3,0 2->5,2 3->5,4 4->4,3,0 5->4,1 6-> 1 7-> 6,5,4,3 8-> x 9-> 4

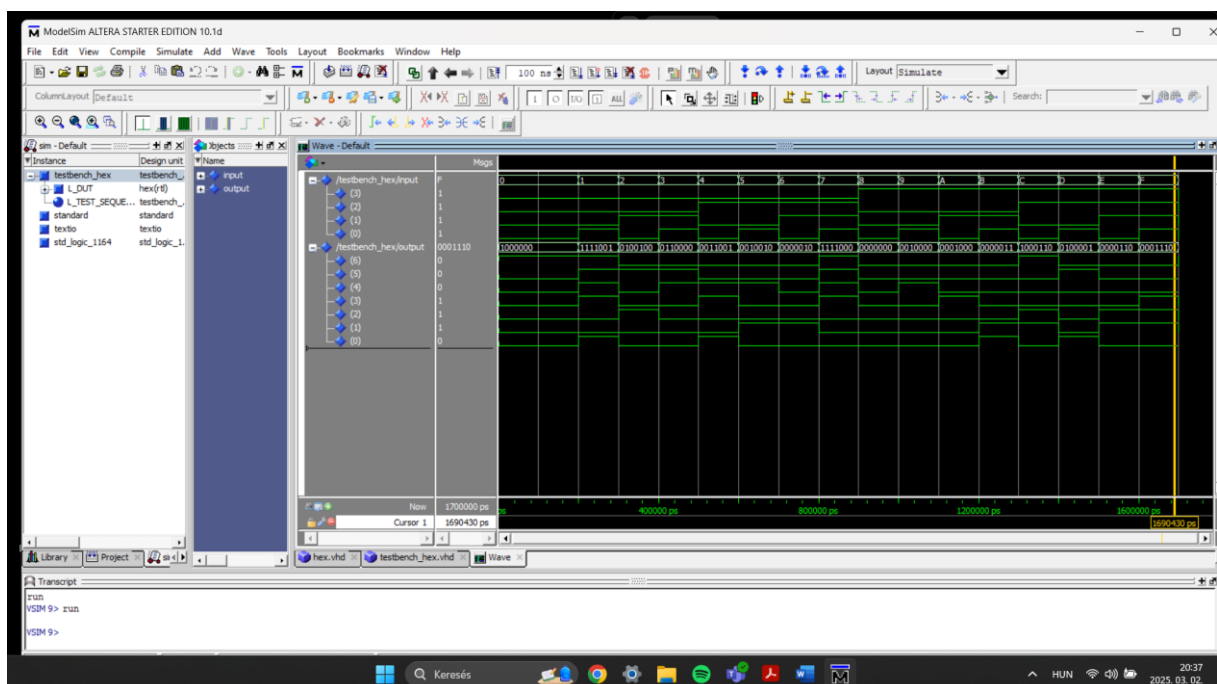
Konverzió:

0000 -> 1000000, 0001 -> 1111001, 0010 -> 0100100, 0011 -> 0110000, 0100 -> 0011001

0101 -> 0010010, 0110 -> 0000010, 0111 -> 1111000, 1000 -> 0000000, 1001 -> 0010000

(A) 1010 -> 0001000, (B) 1011 -> 0000011, (C) 1100 -> 1000110, (D) 1101 -> 0100001

(E) 1110 -> 0000110, (F) 1111 -> 0001110



testbench_hex eredménye

Az rtc_top-ban létrehoztam mindegyik elemet, melyeknek be- és kimeneteit megfelelően összerendeltem.

Bevezettem továbbá a gomb jelét, melyet úgy használok fel, hogy minden órajelben tárolom az előző értékét, így a gomb benyomásakor váltok a két állapot SHOW_TIME (hh:mm) és SHOW_SEC (ss:cc) között.

A szegmensen megjelenítendő értékeket az aktuális állapot szerint választom ki és adom meg a szegmenseknek, felbontva mind a 4 számot egyenként két BCD számjegyre.

testbench_rtc_top eredményei:



Látható, hogy a gomb megnyomása után HEX0 értéke 10^{10} ps-onként változik, ami pontosan 10 ms, 1 századmásodperc. A gomb megnyomása előtt ugyanennyi idő alatt nem változott semmit a HEX0. Ebből látszik, hogy a SHOW_TIME és SHOW_SEC közti állapotváltás megfelelően megtörténik.

Továbbá látszik, hogy a gomb megnyomásakor a HEX0 értéke 0100100 ami megjelenítve 2-t mutat. Ez azért van, mert a gomb megnyomása előtt, már 25 ms eltelt, és ezt számolta is, attól függetlenül, hogy SHOW_TIME módban volt.

A számlálók megfelelő léptetése külön testbench-en is mérve, de itt is megtekinthető, kellő türelemmel.

FPGA megvalósítás

Létrehoztam az új projektet rtc_top néven, hogy a fő VHDL komponenssel könnyű legyen a szinkronizálás. A laboron használt konfigurációval megegyező Cyclone III EP3C16F484C6 eszközt hoztam létre, hozzáadva az elkészített .vhd fájlokat (a testbencheket nem)

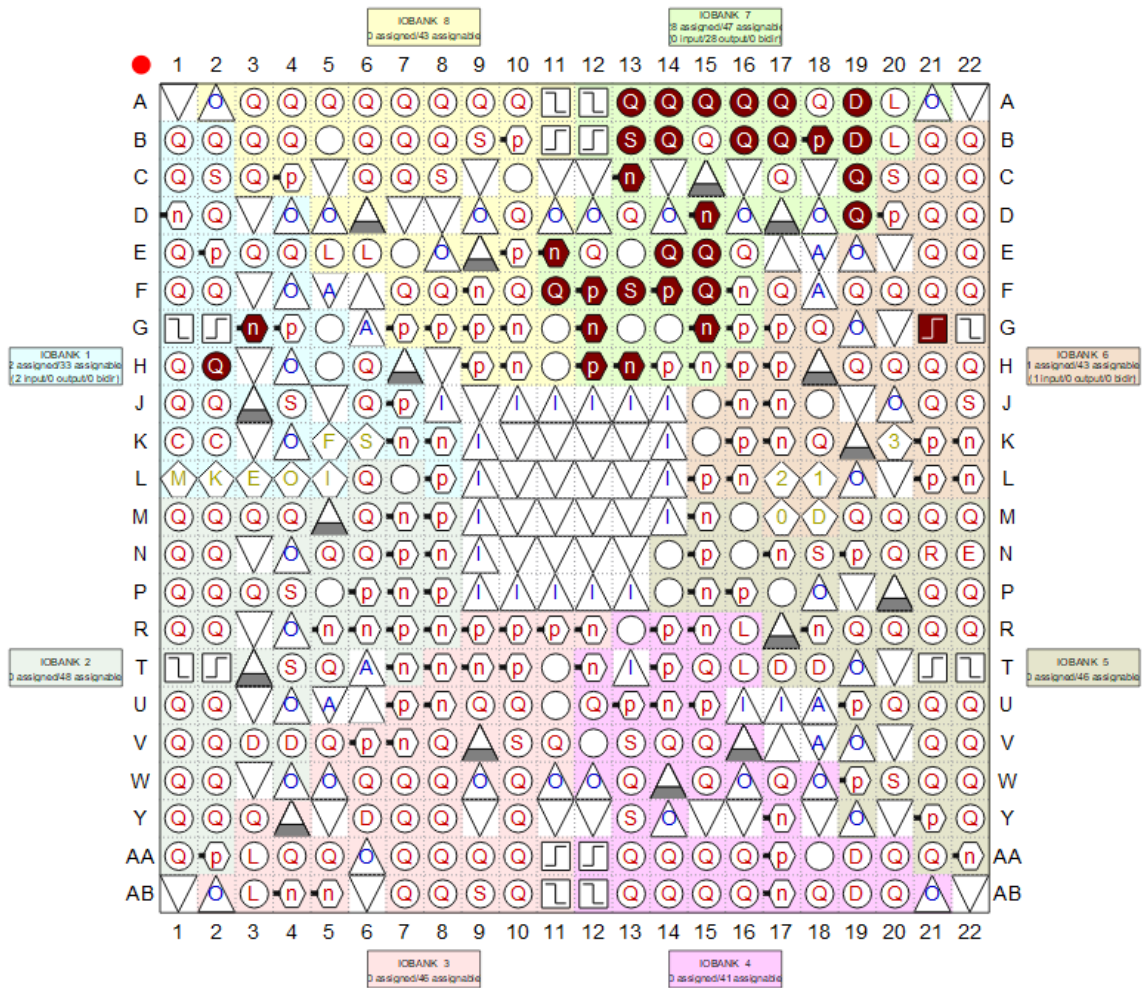
Flow Summary	
Flow Status	Successful - Wed May 07 21:35:41 2025
Quartus II 64-Bit Version	13.1.0 Build 162 10/23/2013 SJ Web Edition
Revision Name	rtc_top
Top-level Entity Name	rtc_top
Family	Cyclone III
Device	EP3C16F484C6
Timing Models	Final
Total logic elements	396
Total combinational functions	396
Dedicated logic registers	46
Total registers	46
Total pins	31
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0

Analysis and Synthesis futtatásának eredménye

Hosszas kutatás után megtaláltam az eszköz dokumentációját (https://www.mouser.com/datasheet/2/598/E0_User_manual_2012-876781.pdf?srltld=AfmBOoqJ3etklD0iveXm_YHjc2ie7dV4ai9-Zjmv7Fno086dQ8YQ_aol) és a 25-29. oldalán a megfelelő pineket kinéztem és a Pin Planner-be beírtam.

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
in_ btn	Input	PIN_G3	1	B1_N0	2.5 V (default)		8mA (default)		
in_ clk	Input	PIN_G21	6	B6_N1	2.5 V (default)		8mA (default)		
out_ HEX0[6]	Output	PIN_F13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX0[5]	Output	PIN_F12	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX0[4]	Output	PIN_G12	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX0[3]	Output	PIN_H13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX0[2]	Output	PIN_H12	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX0[1]	Output	PIN_F11	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX0[0]	Output	PIN_E11	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[6]	Output	PIN_A15	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[5]	Output	PIN_E14	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[4]	Output	PIN_B14	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[3]	Output	PIN_A14	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[2]	Output	PIN_C13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[1]	Output	PIN_B13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX1[0]	Output	PIN_A13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[6]	Output	PIN_F14	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[5]	Output	PIN_B17	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[4]	Output	PIN_A17	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[3]	Output	PIN_E15	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[2]	Output	PIN_B16	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[1]	Output	PIN_A16	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX2[0]	Output	PIN_D15	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[6]	Output	PIN_G15	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[5]	Output	PIN_D19	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[4]	Output	PIN_C19	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[3]	Output	PIN_B19	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[2]	Output	PIN_A19	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[1]	Output	PIN_F15	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
out_ HEX3[0]	Output	PIN_B18	7	B7_N0	2.5 V (default)		8mA (default)	2 (default)	
in_ reset_n	Input	PIN_H2	1	B1_N1	2.5 V (default)		8mA (default)		

Top View - Wire Bond
Cyclone III - EP3C16F484C6

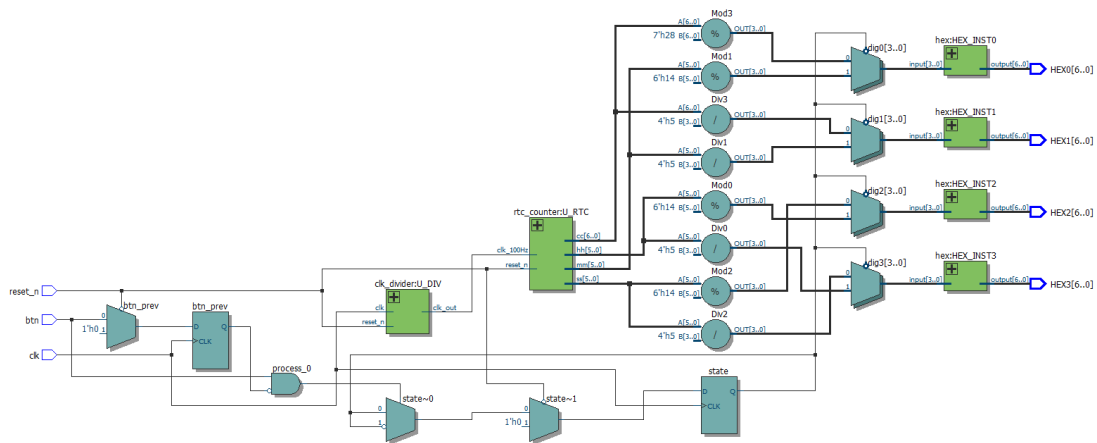


Ezután a Start Compilation gombot megnyomva a fordítás sikeres volt.

```

Type ID Message
204019 Generated file rtc_top_min_1200mv_0c_fast.vho in folder "C:/altera/13.1/imsc/simulation/modelsim/" for EDA simulation tool
204019 Generated file rtc_top.vho in folder "C:/altera/13.1/imsc/simulation/modelsim/" for EDA simulation tool
204019 Generated file rtc_top_6_1200mv_85c_vhd_slow.sdo in folder "C:/altera/13.1/imsc/simulation/modelsim/" for EDA simulation tool
204019 Generated file rtc_top_6_1200mv_0c_vhd_slow.sdo in folder "C:/altera/13.1/imsc/simulation/modelsim/" for EDA simulation tool
204019 Generated file rtc_top_min_1200mv_0c_vhd_fast.sdo in folder "C:/altera/13.1/imsc/simulation/modelsim/" for EDA simulation tool
204019 Generated file rtc_top_vhd.sdo in folder "C:/altera/13.1/imsc/simulation/modelsim/" for EDA simulation tool
> Quartus II 64-Bit EDA Netlist Writer was successful. 0 errors, 0 warnings
293000 Quartus II Full Compilation was successful. 0 errors, 10 warnings

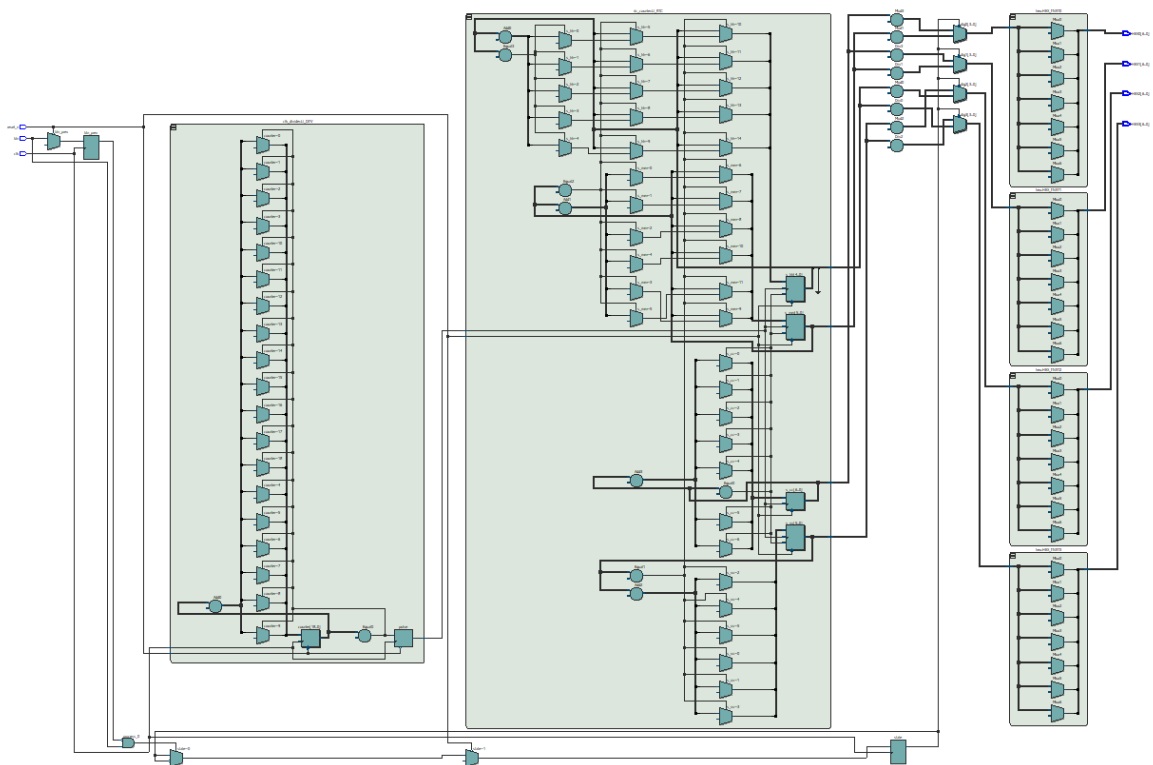
```



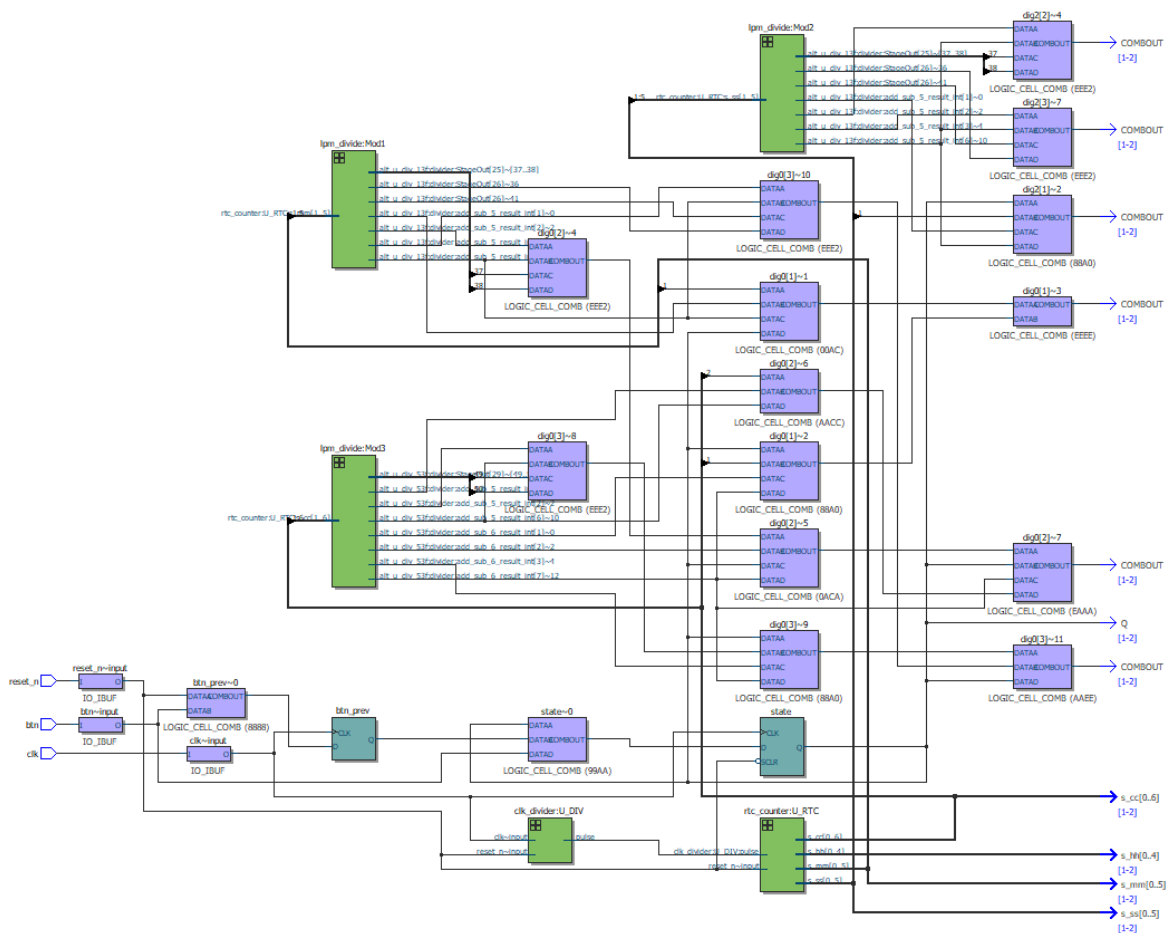
A keletkezett RTL

Mivel nem az egyes időegységek egyes BCD számjegyét kötöttem 1-1 hétszegmentes dekóderre és azoknak a tc-jét rákötni az őt követő enable-re, hanem az egészet egy számlálóban oldottam meg feltételekkel és aritmetikai műveletekkel, ezért a laborfeladattól egészen eltérő terv keletkezett. Ez az esetleg felhasznált logikai egységek számának megnövekedését is eredményezheti, de a feladat még így is kellően egyszerű ahhoz, hogy ez ne okozzon gondot. Az állapotgép is fellelhető.

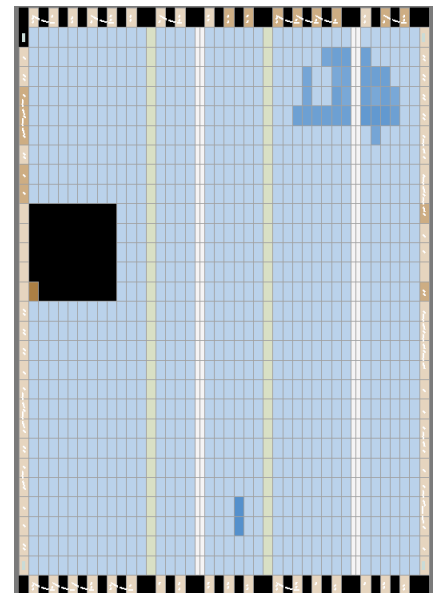
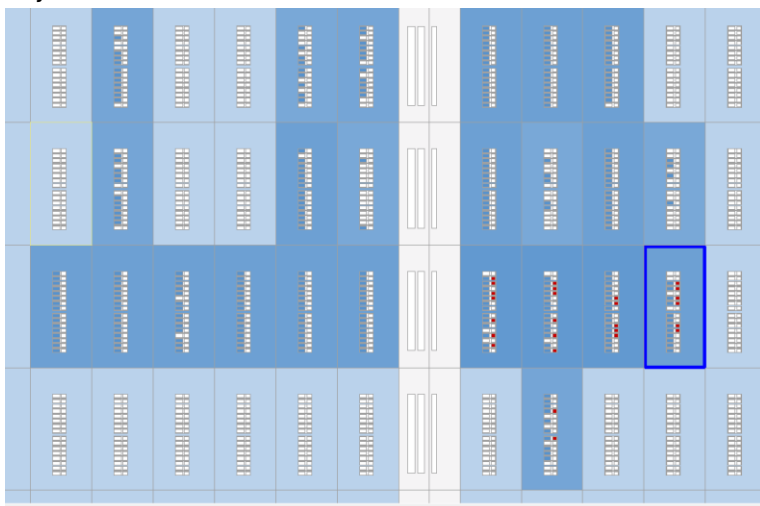
Kíváncsiságból megnéztem a belső működéseket is.



FPGA-ra való szintetizálás, valódi kombinációs hálózatok és LUT-ok használatával:



Chip Planner képe: látható, hogy a laborfeladathoz képest határozottan több logikai egységre van szüksége, bár belenagyítva látszódik, hogy valójában nincsenek ezek teljesen kihasználva.



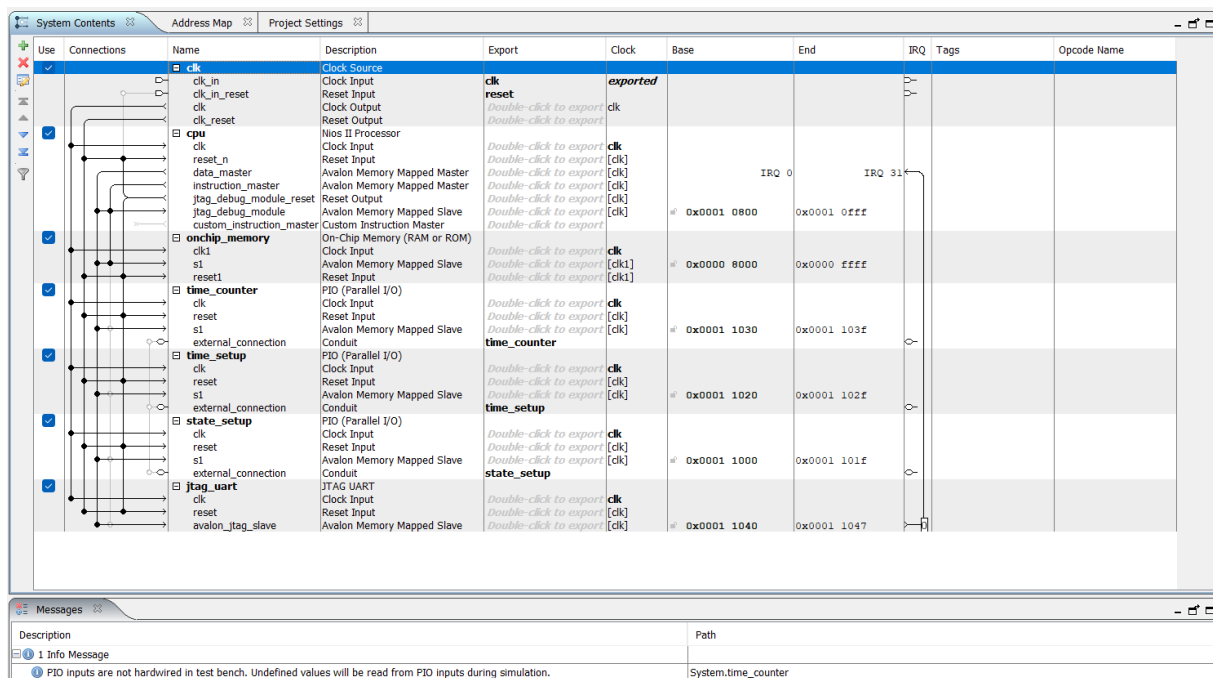
Az FPGA programozására a laboros bemutatáson lesz lehetőség, az output_files mappába generált rtc_top.sof fájl segítségével.

Soft processzoros rendszermegvalósítás

Készítettem biztonsági mentést, valamint létrehoztam egy új revisiont soft_rtc néven.

Buildelés után megnyitottam a Qsys eszközt a virtuális komponensek definiálásához és konfigurálásához. A negyedik laboron hasonló módon ugyanazokat az eszközöket vettem fel. Output-nak egy kétbites state_setup-ot a reset és állapot váltás megvalósításához, valamint egy 32 bites time_setup-ot is a feladatkiírásnak megfelelően. (Valamint az inputot is 32 bitesre állítottam)

A kapcsolatok kialakításámál először az órajelet és a két resetet állítottam be minden eszközre, majd rákötöttem minden eszközt az adatbuszra, a memóriát az utasításbuszra is. Ezután a jtag interruptját és a PIO-k exportját állítottam be, valamint a System → Assign Base Addresses menüponttal az elemeknek osztottam címet. Végezetül beállítottam a cpu-nak a reset és exception vektorát is a memóriára, ezzel megszűnt minden error és figyelmeztetés is.



Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
		clk	Clock Source		exported					
		clk_in	Clock Input	clk						
		clk_in_reset	Reset Input	reset						
		clk	Clock Output	clk						
		clk_reset	Reset Output	reset						
		cpu	Nios II Processor							
		clk	Clock Input	clk						
		reset_n	Reset Input	reset						
		data_master	Avalon Memory Mapped Master	clk						
		instruction_master	Avalon Memory Mapped Master	clk						
		jtag_debug_module_reset	Reset Output	clk						
		jtag_debug_module	Avalon Memory Mapped Slave	clk						
		custom_instruction_master	Custom Instruction Master	clk						
		onchip_memory	On-Chip Memory (RAM or ROM)							
		clk1	Clock Input	clk1						
		s1	Avalon Memory Mapped Slave	clk1						
		reset1	Reset Input	clk1						
		time_counter	PIO (Parallel I/O)	clk						
		clk	Clock Input	clk						
		reset	Reset Input	clk						
		s1	Avalon Memory Mapped Slave	clk						
		time_setup	PIO (Parallel I/O)	time_counter						
		clk	Clock Input	clk						
		reset	Reset Input	clk						
		s1	Avalon Memory Mapped Slave	clk						
		state_setup	PIO (Parallel I/O)	time_setup						
		clk	Clock Input	clk						
		reset	Reset Input	clk						
		s1	Avalon Memory Mapped Slave	clk						
		jtag_uart	JTAG UART	state_setup						
		clk	Clock Input	clk						
		reset	Reset Input	clk						
		avalon_jtag_slave	Avalon Memory Mapped Slave	clk						

Messages

Description	Path
1 Info Message	
PIO inputs are not hardwired in test bench. Undefined values will be read from PIO inputs during simulation.	System.time_counter

Komponensek összekötése

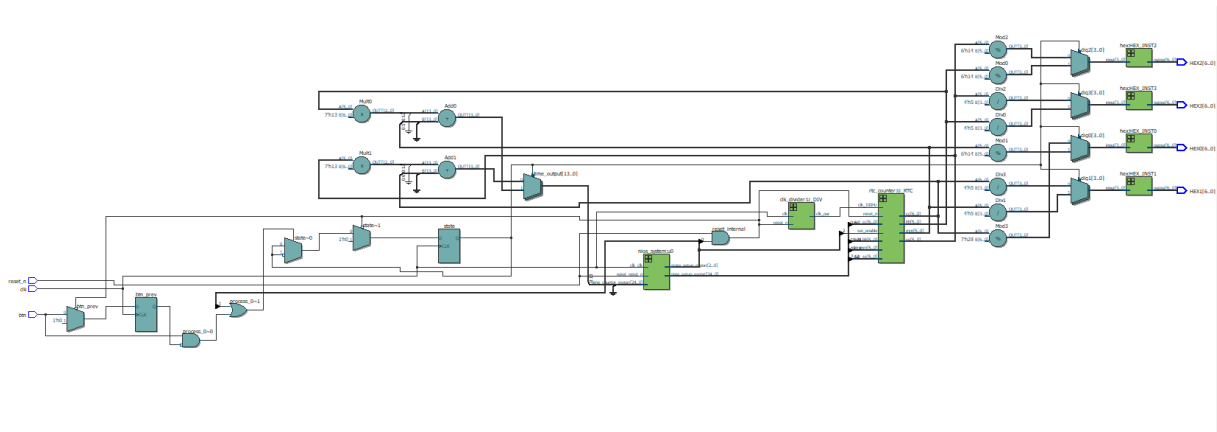
A Generálás után beillesztettem a NIOS Systemet a következőképp:

- Kimásoltam a szükséges VHDL kódot.
- Hozzáadtam a fájlokhoz a nios_system.qsys-t
- A VHDL kódot kiegészítettem a megfelelő működéshez.

Rájöttem, hogy nem volt megfelelő eddig a modellem:

- Az idő PIO-ira elég 25 bit, nem kell 32
- Az idő beállításához kell egy plusz jel is, ezt felvettem a state_setup-ba harmadik bitnek.
- Az idő beállítását az rtc_counter kell hogy végezze, így refaktoráltam egy setterrel és időegység vektorokkal, mint inputok. Ha a set signal 1 akkor beállítja a 4 időegységet megfelelően.

Ezután sikeres volt a fordítás, így következhet a szoftveres rész.



módosult RTL és Analízis

Flow Summary	
Flow Status	Successful - Sun May 11 13:52:03 2025
Quartus II 64-Bit Version	13.1.0 Build 162 10/23/2013 SJ Web Edition
Revision Name	soft_rtc
Top-level Entity Name	rtc_top
Family	Cyclone III
Device	EP3C16F484C6
Timing Models	Final
Total logic elements	2,245
Total combinational functions	2,031
Dedicated logic registers	1,037
Total registers	1037
Total pins	31
Total virtual pins	0
Total memory bits	273,408
Embedded Multiplier 9-bit elements	0
Total PLLs	0

Szoftver implementálása

A NIOS II-höz készített Eclipse Tool-lal létrehoztam egy új projektet BSP-vel, amihez a Hello World Small templatet használtam.

A BSP legenerálása után megnéztem a system.h-ban és a drivers/inc/altera_avalon_pio_regs.h-ban a szükséges makrókat az input-ok és output-ok báziscímeihez, valamint tartalmuk módosításához.

A laboron láttotakhoz hasonlóan készítettem makrókat a vezérlőbitek beállításához: RESET, SWITCH_MODE és SET_TIME.

Ezen funkciók megvalósítására készítettem különálló függvényeket.

A **reset_time** függvény bebillenti a reset bitet, vár 100 makrosekundumot, majd visszaállítja a bitet. A **switch_mode** metódus ugyanezt teszi a switch_mode bitre.

A **set_time** függvény kap egy értéket, amit beállít a time_setup értékévé abban az esetben, ha az egy helyes időt reprezentál. Ezután a megfelelő vezérlőbitet az előzőkhez hasonlóan állítja.

A **read_time** függvény az éppen aktuálisan megjelenített időt kéri le a time counter-en keresztül, majd xx:yy formátumban kiírja az aktuális állapotnak megfelelően.

A **read_for** függvény egy időtartamot kap paraméterül, és ennek az időtartamnak megfelelő időn keresztül 100 ms-onként kiírja az időt a read_time függvény meghívásával.

Készítettem egy **demo** függvényt is, ami bemutatja az RTC működését minden funkció használatával. Reseteli az időt, majd állapotot vált, és átállítja az időt 23 óra 59 perc 30 másodperc 30 ms-re. Az utasítások között rendszeresen lekérdezi a mért időt.

A **main** függvényemet úgy készítettem el, hogy JTAG UART-on terminálból szöveges paranccsal vezérelhesse a felhasználó az órát.

Így tehát definiáltam reset, mode, set <hhmmsscc>, read, readfor <t> és demo parancsokat a különböző funckiók eléréséhez. A parancsokat végtelen ciklusban olvasom be, és a bemenetnek megfelelő parancsot végrehajtom. Ehhez készítettem segédfüggvényeket:

read_line-nal beolvasok egy sort, **starts_with**-del leellenőrzöm, hogy a beolvasott sor melyik parancsnak felel meg, read_num-mal a beírt számokra konvertálom a beolvasott karaktertömböt.